

ObservaAção — Ouvidoria Cidadã Digital

1. Overview — Painel de Qualidade

O SonarCloud analisou o projeto ObservaAção no branch **main**. A imagem abaixo apresenta o painel-resumo e a seguir o detalhamento das métricas coletadas.

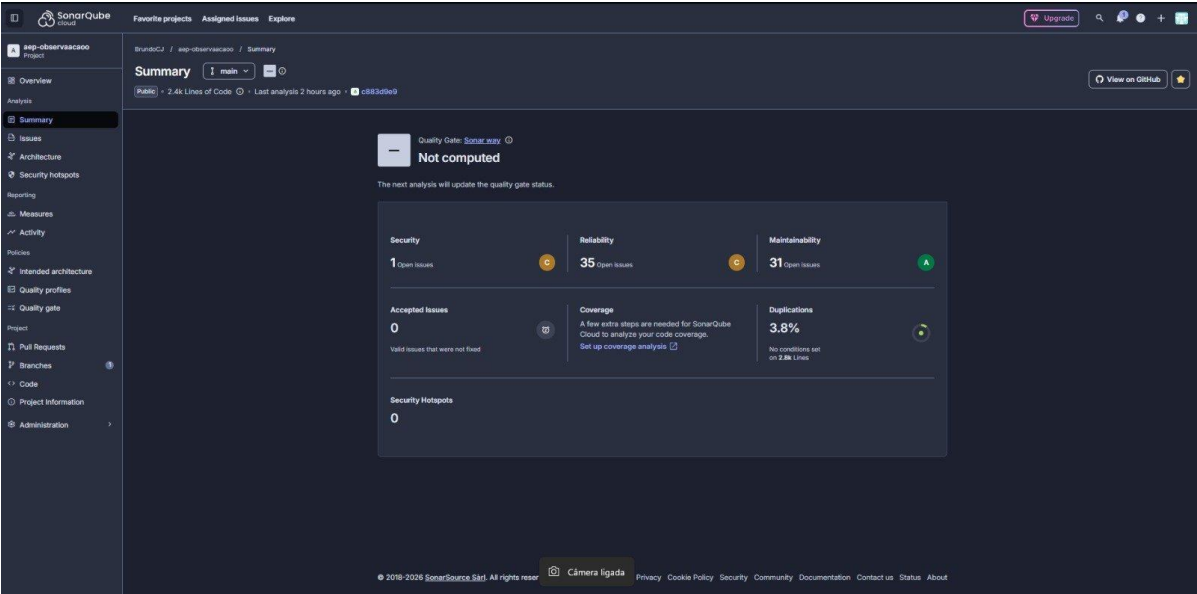


Figura 1 — Painel Summary do SonarCloud (branch main)

Métrica	Resultado
Linhas de código analisadas	2.4k (2.8k incl. comentários e espaços)
Security	1 issue aberto — Grau C (vulnerabilidade Major)
Reliability	35 issues abertos — Grau C
Maintainability	31 issues abertos — Grau A
Duplicações	3.8% do código
Coverage	Não configurada (requer setup do JaCoCo)
Security Hotspots	0 identificados

2. Maintainability — Manutenibilidade

Avalia a facilidade de compreender, modificar e evoluir o código. O SonarCloud identificou **31 issues**. O projeto recebeu **Grau A** (o melhor possível), indicando débito técnico baixo em relação ao tamanho do código.

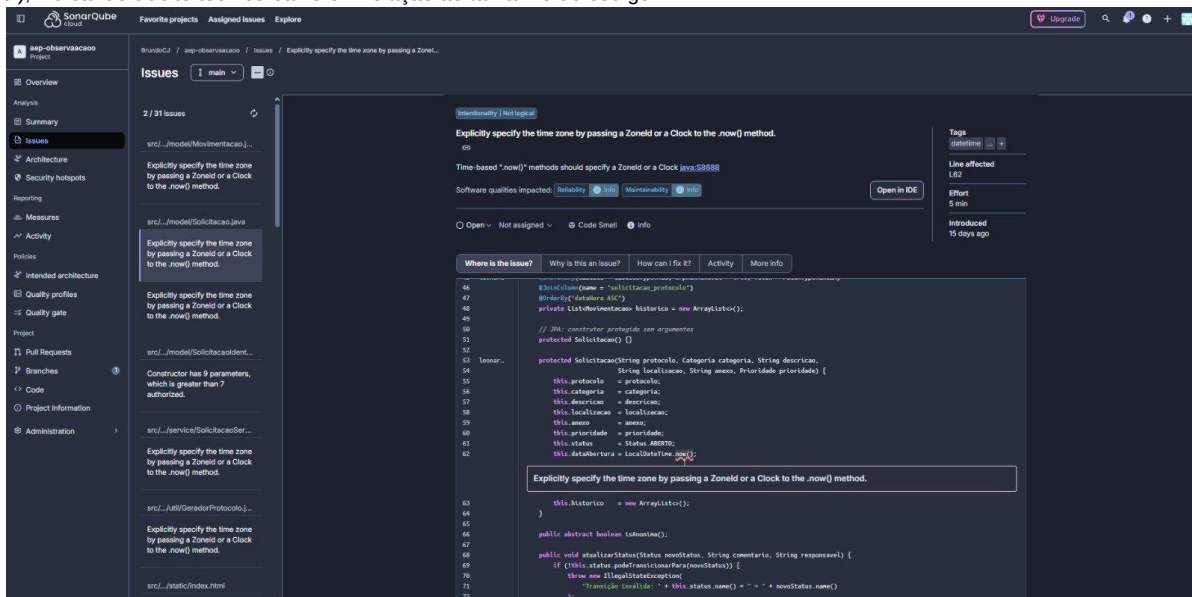


Figura 2 — Issues de manutenibilidade detectadas pelo SonarCloud

Caso 1 — `LocalDateTime.now()` sem `ZoneId` especificado

Code Smell · Severidade: Info · Esforço: 5 min por ocorrência

Arquivos afetados: `Movimentacao.java`, `Solicitacao.java`, `SolicitacaoService.java`, `GeradorProtocolo.java`

Como isso impacta:

- `LocalDateTime.now()` usa o fuso padrão da JVM, que varia entre dev e produção.
- Em servidores cloud em UTC, a data ficaria 3 horas adiantada em relação ao horário de Brasília.
- Pode gerar inconsistências no cálculo do prazo SLA e no histórico de movimentações.
- A intenção do código não é explícita, dificultando manutenção por outros desenvolvedores.

Como resolver:

- Substituir por `LocalDateTime.now(ZoneId.of("America/Sao_Paulo"))` em todos os pontos.
- Criar constante `ZONE_BR = ZoneId.of("America/Sao_Paulo")` reutilizável.
- Para ambientes UTC: usar `ZoneId.of("UTC")` e converter na camada de apresentação.

Caso 2 — Construtor com mais de 7 parâmetros

Code Smell · Severidade: Info · Esforço: 30 min

Arquivo afetado: `SolicitacaoIdentificada.java` — construtor com 9 parâmetros (limite: 7).

Como isso impacta:

- 9 argumentos é difícil de ler, chamar corretamente e testar.
- Ordem de parâmetros pode ser trocada acidentalmente sem detecção do compilador.
- Novos campos exigem alterar o construtor e todos os locais que o chamam.
- Indica acúmulo de responsabilidades — possível violação do SRP.

Como resolver:

- Aplicar o padrão **Builder** com métodos encadeados.
- Criar Value Object `DadosPessoais(nome, email, telefone)` agrupando campos relacionados.
- Construtor final: `SolicitacaoIdentificada(protocolo, dadosPessoais, dadosOcorrencia)`.
- DTOs de request já aplicam parte dessa separação — refletir no domínio é o próximo passo.

3. Reliability — Confiabilidade

Analisa a probabilidade de o sistema se comportar incorretamente em execução. Com **35 issues** e **Grau C**, é a área que merece maior atenção. A maioria dos issues está relacionada ao uso de datas e horários sem timezone explícito.

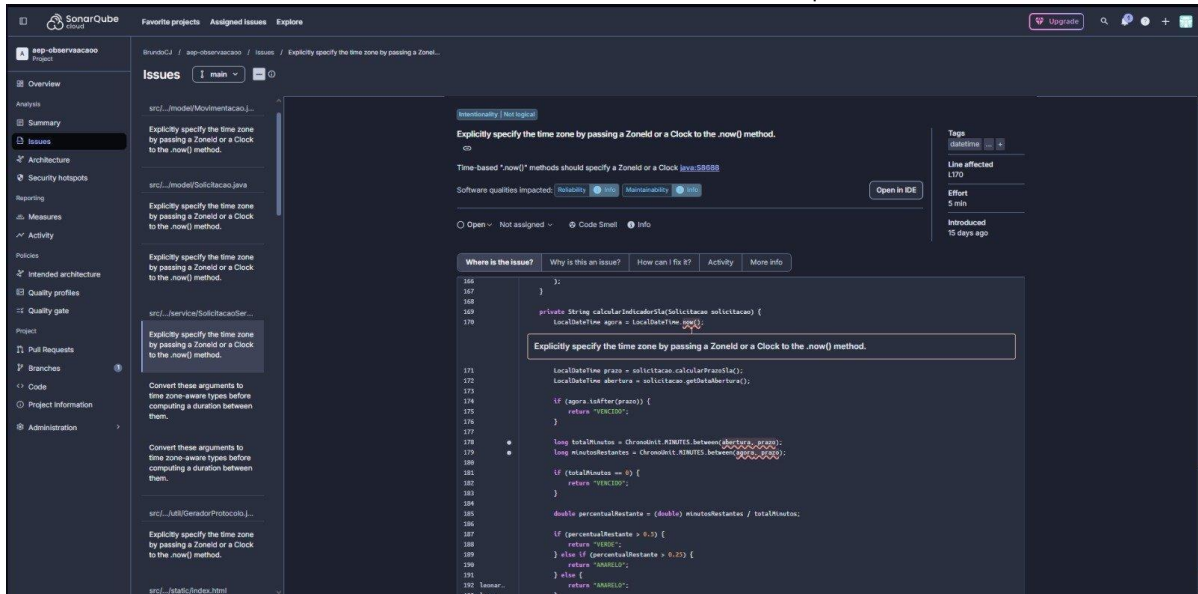


Figura 3 — Cálculo de SLA usando ChronoUnit sem tipos conscientes de timezone

Caso 1 — `LocalDateTime.now()` afetando a confiabilidade dos dados

Code Smell · Impacto: Reliability + Maintainability · Esforço: 5 min/ocorrência

Arquivos afetados: Solicitacao.java (L62), SolicitacaoService.java.

Como isso impacta:

- `dataAbertura` pode ser inconsistente entre dev (horário local) e prod (UTC em cloud).
- Cálculo do prazo SLA (`dataAbertura.plusDays(slaDias)`) só é correto com timezone consistente.
- Prazo incorreto pode fazer atendente crer estar no prazo quando já está vencido.
- Em AWS/GCP/Azure (operam em UTC), o impacto é amplificado.

Como resolver:

- Padronizar `ZonedDateTime/OffsetDateTime` no domínio.
- Forma simples: `LocalDateTime.now(ZoneId.of("America/Sao_Paulo"))`.
- Configurar `spring.jackson.time-zone=America/Sao_Paulo` no `application.properties`.

Caso 2 — Cálculo de duração sem tipos conscientes de timezone

Code Smell · Impacto: Reliability · Esforço: 15 min

Arquivo afetado: SolicitacaoService.java — `calcularIndicadorSla()` usa `ChronoUnit.MINUTES.between()` com `LocalDateTime` sem `ZoneId`.

Como isso impacta:

- `ChronoUnit.MINUTES.between(abertura, prazo)` só funciona se ambos no mesmo timezone.
- Em transições de DST, 60 minutos podem ser calculados como 59 ou 61.
- Indicador SLA (VERDE/AMARELO/VENCIDO) pode ser exibido errado, afetando priorização.
- Percentual restante do prazo fica impreciso, podendo alterar estados sem refletir a realidade.

Como resolver:

- Substituir por `ZonedDateTime.now(ZoneId.of("America/Sao_Paulo"))`.
- Usar `ChronoUnit.MINUTES.between(abertura.toInstant(), prazo.toInstant())` (UTC).
- Ou comparar via `epochSecond`. Garante SLA correto independente de DST/config do servidor.

4. Security — Segurança

Identifica vulnerabilidades que podem comprometer integridade, confidencialidade ou disponibilidade. Com **1 issue Major e Grau C**, o projeto possui um ponto crítico a corrigir antes de qualquer exposição pública ou execução em produção.

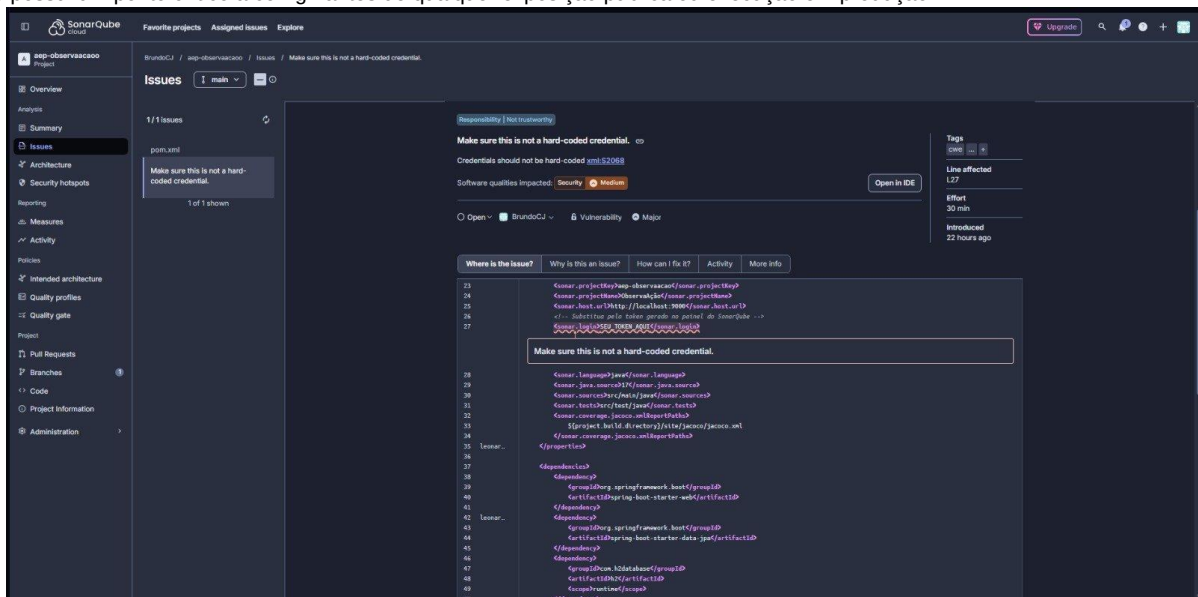


Figura 4 — Vulnerabilidade CWE-798: credencial hard-coded detectada no pom.xml

Caso 1 — Credencial hard-coded no pom.xml

Vulnerabilidade · Severidade: Major · CWE-798 · Esforço: 30 min

Arquivo afetado: pom.xml — linha 27 — `<sonar.login>SEU_TOKEN_AQUI</sonar.login>`

Como isso impacta:

- Token real ficaria exposto publicamente — o repositório é público no GitHub.
- CWE-798 é uma das vulnerabilidades mais exploradas; qualquer um com acesso ao repo usaria o token.
- Token comprometido permite alterar quality gates e suprimir issues no SonarCloud.
- Estabelece padrão ruim: outros devs podem inserir credenciais reais seguindo o template.

Como resolver:

- Remover completamente a propriedade `<sonar.login>` do pom.xml.
- Passar o token só na análise: `.mvnw.cmd sonar:sonar -Dsonar.login=TOKEN`.
- Em CI/CD GitHub Actions: usar `secrets.SONAR_TOKEN` — nunca commitar o valor.
- Adicionar `sonar.properties` ao .gitignore.

Considerações Finais

A análise evidencia uma base de código bem organizada: **Grau A em Maintainability** com apenas **3.8% de duplicação**. Os pontos de melhoria concentram-se no tratamento de datas sem timezone explícito — issue de maior recorrência em múltiplos arquivos e categorias. A vulnerabilidade de segurança é de correção simples e deve ser resolvida antes da próxima análise para que o Quality Gate compute corretamente. Com as correções aplicadas, o projeto tem condições de atingir **Grau B ou superior** nas três categorias.

Prioridade	Ação
CRÍTICO	Remover sonar.login do pom.xml (CWE-798)
ALTA	Substituir <code>LocalDateTime.now()</code> por <code>LocalDateTime.now(Zoneld...)</code> em todos os arquivos
MÉDIA	Corrigir <code>ChronoUnit.MINUTES.between()</code> para usar tipos conscientes de timezone
BAIXA	Refatorar <code>SolicitacaoIdentificada</code> para reduzir parâmetros do construtor
FUTURO	Configurar coverage com JaCoCo e criar testes unitários (src/test ainda ausente)